

04 — Boolean & UUID Types in PostgreSQL

Table of Contents

1. [Learning Objectives](#)
 2. [Boolean Type](#)
 3. [UUID Type](#)
 4. [UUID Generation Strategies](#)
 5. [UUID vs BIGINT as Primary Keys](#)
 6. [UUID Variants Explained](#)
 7. [Storage Reference](#)
 8. [SQL Examples](#)
 9. [Common Mistakes](#)
 10. [Best Practices](#)
 11. [Performance Considerations](#)
 12. [Interview Questions & Answers](#)
 13. [Exercises with Solutions](#)
 14. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Use the boolean type correctly, including its three-valued logic (TRUE/FALSE/NULL)
 - Generate and store UUIDs efficiently
 - Make informed decisions between UUID and BIGINT primary keys
 - Understand UUID versions and choose the right one for your use-case
 - Avoid common UUID indexing performance traps
-

Boolean Type

Definition

PostgreSQL's `BOOLEAN` type stores a truth value: `TRUE` , `FALSE` , or `NULL` .

- **Storage:** 1 byte
- **Values:** `TRUE` , `FALSE` , `NULL` (unknown)

Accepted input literals

```
-- TRUE values:
TRUE, 't', 'true', 'y', 'yes', 'on', '1'

-- FALSE values:
FALSE, 'f', 'false', 'n', 'no', 'off', '0'

-- These are case-insensitive
SELECT 'Yes'::BOOLEAN; -- TRUE
SELECT 'OFF'::BOOLEAN; -- FALSE
SELECT 'TRUE'::BOOLEAN; -- TRUE
```

Three-valued logic

SQL uses three-valued logic: TRUE, FALSE, and NULL (unknown).

AND truth table:

	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

OR truth table:

	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

NOT truth table:

NOT TRUE = FALSE
NOT FALSE = TRUE
NOT NULL = NULL

Boolean in queries

```
-- Filter on boolean column
SELECT * FROM users WHERE is_active;           -- is_active = TRUE
SELECT * FROM users WHERE NOT is_active;       -- is_active = FALSE
SELECT * FROM users WHERE is_active IS NULL;   -- is_active = NULL
SELECT * FROM users WHERE is_active IS NOT NULL; -- is_active is set

-- Common mistake: = NULL never matches
SELECT * FROM users WHERE is_active = NULL;   -- returns 0 rows (always!)
SELECT * FROM users WHERE is_active IS NULL;  -- correct
```

Boolean aggregations

```
-- Count TRUE values
SELECT count(*) FILTER (WHERE is_active) AS active_count FROM users;

-- bool_and: TRUE if all values are TRUE
-- bool_or: TRUE if any value is TRUE
SELECT
    bool_and(is_active) AS all_active,
    bool_or(is_active)  AS any_active
FROM users;

-- Convert boolean to integer for sums
SELECT
    sum(CASE WHEN is_active THEN 1 ELSE 0 END) AS active_int_count
FROM users;
```

```
-- Or more cleanly:  
SELECT sum(is_active::INTEGER) AS active_count FROM users;
```

UUID Type

Definition

UUID (Universally Unique Identifier) is a 128-bit number formatted as:

XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX
8 4 4 4 12 (hex digits)

- **Storage:** 16 bytes (stored as raw binary, not as the 36-char string)
- **Number of possible values:** $2^{128} \approx 3.4 \times 10^{38}$ (practically inexhaustible)

UUID format

[illegible]

UUID Generation Strategies

uuid-oss extension (classic)

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

SELECT uuid_generate_v1();    -- v1: time + MAC address (sequential, exposes MAC)
SELECT uuid_generate_v1mc(); -- v1 with random multicast MAC
SELECT uuid_generate_v3(uuid_ns_url(), 'https://example.com'); -- v3: MD5 hash
SELECT uuid_generate_v4();    -- v4: fully random (most common)
SELECT uuid_generate_v5(uuid_ns_url(), 'https://example.com'); -- v5: SHA-1 hash
```

Built-in gen_random_uuid() (PostgreSQL 13+)

```
-- No extension required! Uses pgcrypto internally
SELECT gen_random_uuid(); -- generates UUIDv4
-- Returns: a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11
```

UUID Version Comparison

UUID Versions

- v1: time-based + MAC address
 - ✓ Sequential within same host

- ✗ Exposes server MAC address (privacy risk)
- ✗ Not sortable across nodes

v3: MD5 hash of namespace + name

- ✓ Deterministic (same inputs → same UUID)
- ✓ Good for content-addressed identifiers
- ✗ MD5 is cryptographically weak

v4: Fully random (122 random bits)

- ✓ No information leakage
- ✓ Most widely used
- ✗ Not sequential → random index inserts → B-tree fragmentation

v5: SHA-1 hash of namespace + name

- ✓ Deterministic
- ✓ Better than v3 cryptographically
- ✗ Still not sequential

v7: Unix timestamp + random (PostgreSQL 17 / pgcrypto)

- ✓ Sequential – timestamp in high bits
- ✓ Sortable by creation time
- ✓ No information leakage
- ✓ Best of all worlds for PKs

UUIDv7 (modern, recommended for PKs)

```
-- PostgreSQL 17+: built-in
SELECT uuidv7();

-- Or via extension for earlier versions:
CREATE EXTENSION IF NOT EXISTS pg_uuidv7;
SELECT uuid_generate_v7();
-- Returns: 018e7e24-3b80-7123-abcd-ef0123456789
-- First 48 bits = Unix timestamp in ms → sortable!
```

UUID vs BIGINT as Primary Keys

Comparison: UUID vs BIGINT

	BIGINT	UUID (v4)	UUID (v7)
Storage (PK column)	8 bytes	16 bytes	16 bytes
Storage (FK column)	8 bytes	16 bytes	16 bytes
Index entry size	~50 bytes	~70 bytes	~70 bytes
Sequential inserts	Yes	No	Yes
Merge/shard safe	No*	Yes	Yes
Guessable	Yes	No	No
Human readable	Yes	No	No
Sort by creation time	Yes	No	Yes

URL safe (as-is)	Yes	Yes	Yes
* BIGINT identity columns are per-table; merge requires UUID or manual coordination			

When to use UUID

- Distributed systems where rows are created on multiple nodes
- IDs exposed in URLs/APIs (not guessable: 2¹²² vs sequential)
- When merging data from multiple databases
- Multi-tenant SaaS where tenants may generate IDs independently
- Event sourcing / CQRS architectures

When to use BIGINT

- Single-node databases with high write throughput
- Tables that will join frequently (smaller key = faster joins)
- When humans need to reference IDs verbally or in support tickets
- Internal tables never exposed in APIs

UUID Variants Explained

Byte layout of UUID (16 bytes):

Offset	Description
0-3	time_low (v1) or random (v4)
4-5	time_mid (v1) or random (v4)
6-7	version (4 bits) + time_hi (v1) or random (v4)
8	variant (2 bits) + clock_seq_hi (v1) or random (v4)
9	clock_seq_low (v1) or random (v4)
10-15	node (MAC or random) (v1) or random (v4)

Version field (bits 4-7 of byte 6):

- 0001 = version 1
- 0011 = version 3
- 0100 = version 4
- 0101 = version 5
- 0111 = version 7

Variant field (bits 0-1 of byte 8):

- 10xx = RFC 4122 (standard)
- 110x = Microsoft legacy
- 111x = reserved

Storage Reference

Type	Storage	Range/Values	Notes
boolean	1 byte	TRUE, FALSE, NULL	3-valued logic
uuid	16 bytes	2 ¹²⁸ unique values	Stored as binary, not 36-char str

UUID storage efficiency

VARCHAR(36) for '550e8400-e29b-41d4-a716-446655440000':
36 bytes + 1 overhead = 37 bytes (plus alignment)

UUID type:
16 bytes (no hyphen overhead, pure binary)

For a table with 100M rows and a UUID FK:
VARCHAR(36): 3.7 GB for that column
UUID: 1.6 GB for that column (57% smaller!)

SQL Examples

Complete user system with boolean flags and UUID PK

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";

CREATE TABLE users (
  user_id      UUID          NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
  email        TEXT          NOT NULL UNIQUE,
  username     VARCHAR(50)   NOT NULL UNIQUE,

  -- Boolean status flags
  is_active    BOOLEAN       NOT NULL DEFAULT TRUE,
  is_verified  BOOLEAN       NOT NULL DEFAULT FALSE,
  is_admin     BOOLEAN       NOT NULL DEFAULT FALSE,
  email_confirmed BOOLEAN     NOT NULL DEFAULT FALSE,

  -- Nullable boolean: three states (yes/no/not answered)
  agreed_to_marketing BOOLEAN, -- NULL = not asked yet

  created_at   TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at   TIMESTAMPTZ   NOT NULL DEFAULT now(),
  last_login   TIMESTAMPTZ

);

-- Feature flags table
CREATE TABLE feature_flags (
  flag_id      UUID          NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
  flag_name    TEXT          NOT NULL UNIQUE,
  is_enabled   BOOLEAN NOT NULL DEFAULT FALSE,
  description  TEXT,
  updated_at   TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- User-feature flag overrides (many-to-many)
CREATE TABLE user_feature_flags (
  user_id      UUID          NOT NULL REFERENCES users(user_id),
  flag_name    TEXT          NOT NULL REFERENCES feature_flags(flag_name),
```

```

        is_enabled BOOLEAN NOT NULL,
        set_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
        PRIMARY KEY (user_id, flag_name)
    );

-- Check if a user has a feature enabled
-- (user override takes precedence over global flag)
CREATE OR REPLACE FUNCTION user_has_feature(p_user_id UUID, p_flag TEXT)
RETURNS BOOLEAN
LANGUAGE SQL STABLE AS $$
    SELECT COALESCE(
        (SELECT is_enabled FROM user_feature_flags
         WHERE user_id = p_user_id AND flag_name = p_flag),
        (SELECT is_enabled FROM feature_flags WHERE flag_name = p_flag),
        FALSE
    );
$$;

-- Sessions table with UUID
CREATE TABLE sessions (
    session_id    UUID          NOT NULL DEFAULT gen_random_uuid() PRIMARY KEY,
    user_id       UUID          NOT NULL REFERENCES users(user_id),
    is_active     BOOLEAN       NOT NULL DEFAULT TRUE,
    ip_address    INET,
    user_agent    TEXT,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
    expires_at    TIMESTAMPTZ NOT NULL DEFAULT now() + INTERVAL '30 days',
    last_activity TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Index for session lookups
CREATE INDEX idx_sessions_user ON sessions (user_id) WHERE is_active;
CREATE INDEX idx_sessions_expiry ON sessions (expires_at) WHERE is_active;

```

Querying with boolean logic

```

-- Deactivate expired sessions
UPDATE sessions
SET is_active = FALSE
WHERE is_active = TRUE
    AND expires_at < now();

-- Find users who need email confirmation reminder
SELECT user_id, email, created_at
FROM users
WHERE is_active = TRUE
    AND email_confirmed = FALSE
    AND created_at < now() - INTERVAL '3 days';

-- Summary statistics using boolean aggregation
SELECT

```

```

count(*) AS total_users,
count(*) FILTER (WHERE is_active) AS active_users,
count(*) FILTER (WHERE is_verified) AS verified_users,
count(*) FILTER (WHERE is_admin) AS admin_users,
count(*) FILTER (WHERE agreed_to_marketing IS NULL) AS not_asked_marketing,
count(*) FILTER (WHERE agreed_to_marketing = TRUE) AS marketing_opted_in,
ROUND(100.0 * sum(is_active::INT) / count(*), 2) AS pct_active
FROM users;

```

Common Mistakes

1. Comparing boolean with `= NULL` instead of `IS NULL`

```

-- BAD: returns 0 rows always
WHERE agreed_to_marketing = NULL

-- GOOD
WHERE agreed_to_marketing IS NULL

```

2. Storing UUIDs as VARCHAR(36)

```

-- BAD: 37 bytes, string comparison, no type safety
id VARCHAR(36) PRIMARY KEY

-- GOOD: 16 bytes, binary comparison, type-safe
id UUID PRIMARY KEY DEFAULT gen_random_uuid()

```

3. Using UUIDv4 for primary keys without understanding index fragmentation — Random UUIDs cause B-tree page splits on every insert. Use UUIDv7 or BIGINT identity for high-write tables.

4. NOT NULL boolean columns without a default

```

-- BAD: requires explicit value on every insert
is_active BOOLEAN NOT NULL

-- GOOD: sensible default
is_active BOOLEAN NOT NULL DEFAULT TRUE

```

5. Using integer 0/1 instead of boolean

```

-- BAD: loses semantic meaning, allows values other than 0 and 1
is_active SMALLINT

-- GOOD: enforced by type system
is_active BOOLEAN NOT NULL DEFAULT TRUE

```

6. UUID comparison performance oversight — UUID equality comparison is fast (binary). UUID ordering/sorting is lexicographic (not chronological for v4). For time-sorted UUIDs, use v7.

Best Practices

1. Use **BOOLEAN** for binary states — never SMALLINT, CHAR(1), or INTEGER.
2. Use **three-valued logic intentionally** — a NULL boolean can represent "not yet known" which is semantically different from FALSE.
3. Use **gen_random_uuid()** (PostgreSQL 13+) for v4 UUIDs — no extension required.
4. **Prefer UUIDv7** for primary keys in new distributed systems — sequential and private.
5. Use **BIGINT GENERATED AS IDENTITY** for single-node high-write tables where UUIDs are not needed.
6. **Never use VARCHAR(36) for UUIDs** — always use the native **UUID** type.
7. **Index boolean columns sparingly** — low-cardinality columns have poor index selectivity unless combined with other columns or using partial indexes.
8. **Use partial indexes on boolean flags:**

```
CREATE INDEX idx_users_unverified ON users (created_at)
WHERE email_confirmed = FALSE; -- only indexes unverified users
```

Performance Considerations

Boolean indexing

```
-- Low selectivity: B-tree index on boolean is usually not useful
-- (if 70% of rows are TRUE, the index is rarely chosen by the planner)

-- GOOD: partial index - only indexes the minority case
CREATE INDEX idx_unconfirmed ON users (user_id)
WHERE email_confirmed = FALSE;

-- GOOD: composite index - boolean + another column
CREATE INDEX idx_active_users ON users (created_at)
WHERE is_active = TRUE;
```

UUID B-tree fragmentation with v4

Sequential BIGINT inserts:

Always append to the rightmost leaf node

1	2	3	4
---	---	---	---

 ← clean, sequential

Random UUID v4 inserts:

Each insert splits a random page

a7f...	3c2...	89d...
--------	--------	--------

 ← scattered, causes page splits

Result: index bloat, more I/O, slower inserts

UUIDv7 inserts:

Timestamp prefix makes them sequential

018e7..	018e8..	018e9..	← sequential, like BIGINT
---------	---------	---------	---------------------------

Benchmark: insert throughput (approximate)

BIGINT IDENTITY:	~200,000 inserts/sec (sequential, no fragmentation)
UUID v7:	~180,000 inserts/sec (near-sequential)
UUID v4:	~80,000 inserts/sec (random, page splits, bloat)

Interview Questions & Answers

Q1: What are the three possible values of a boolean in PostgreSQL?

A: TRUE, FALSE, and NULL. SQL uses three-valued logic. NULL represents "unknown" — it is not the same as FALSE. This is why `WHERE flag = NULL` never matches: the comparison of anything with NULL is NULL, not TRUE or FALSE.

Q2: How does PostgreSQL store UUID values internally?

A: As 16 bytes of raw binary data. Despite the hyphenated text representation (`xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`) being 36 characters, the stored value is 16 bytes. This makes UUID storage significantly more compact than VARCHAR(36) and comparisons are pure binary operations.

Q3: What is the difference between UUID v4 and v7?

A: UUIDv4 uses 122 bits of randomness — completely unpredictable and private, but unordered (causing B-tree fragmentation). UUIDv7 encodes a Unix millisecond timestamp in the high bits followed by random bits — this makes v7 UUIDs naturally sortable by creation time while still being effectively unique and non-guessable.

Q4: Why can UUIDv4 cause B-tree index performance problems?

A: B-tree indexes work most efficiently when new values are inserted at the end (highest value). UUIDv4 values are random, so each insert lands in a random position in the B-tree, causing page splits throughout the tree. This leads to index bloat (fragmentation) and slower insert performance compared to sequential values.

Q5: When would you use a nullable boolean instead of a non-null boolean?

A: When three states are semantically meaningful. Example: `agreed_to_marketing` can be NULL (user hasn't been asked), TRUE (agreed), or FALSE (declined). A non-null boolean with default FALSE would conflate "not asked" with "declined".

Q6: How would you migrate a table from SERIAL to UUID primary key?

A: Add a new UUID column, backfill with `gen_random_uuid()`, update all foreign keys, drop the old column, rename the new one. In practice, this requires a maintenance window or use of `pg_repack` for zero-downtime migration.

Q7: What does `bool_and()` return if all rows are NULL?

A: NULL. Aggregate functions that encounter only NULLs return NULL (except COUNT which returns 0).

Q8: How do you generate a deterministic UUID from a known input?

A: Use UUID v5 (SHA-1 based) or v3 (MD5 based) with a namespace: `uuid_generate_v5(uuid_ns_url(), 'https://example.com/resource/123')`. The same inputs always produce the same UUID — useful for content-addressed identifiers or idempotent imports.

Exercises with Solutions

Exercise 1

Create a `feature_toggles` system where users can have per-user overrides of global feature flags. Use UUID PKs, boolean flags, and appropriate indexes.

Solution: (see the complete example in SQL Examples section above)

Exercise 2

Write a query that shows, for each user, whether they have all required profile fields completed (name, email, bio, avatar_url) as a single boolean column `profile_complete`.

Solution:

```
SELECT
  user_id,
  email,
  (
    full_name IS NOT NULL AND length(trim(full_name)) > 0
    AND email IS NOT NULL
    AND bio IS NOT NULL AND length(trim(bio)) > 10
    AND avatar_url IS NOT NULL
  ) AS profile_complete
FROM users;
```

Exercise 3

Convert a table that uses CHAR(1) 'Y'/'N' booleans to use the native BOOLEAN type.

Solution:

```
-- Step 1: Add new column
ALTER TABLE users ADD COLUMN is_subscribed_new BOOLEAN;

-- Step 2: Migrate data
UPDATE users SET is_subscribed_new = (is_subscribed_old = 'Y');

-- Step 3: Set NOT NULL and default
ALTER TABLE users ALTER COLUMN is_subscribed_new SET NOT NULL;
ALTER TABLE users ALTER COLUMN is_subscribed_new SET DEFAULT FALSE;

-- Step 4: Drop old column
ALTER TABLE users DROP COLUMN is_subscribed_old;
```

```
-- Step 5: Rename
ALTER TABLE users RENAME COLUMN is_subscribed_new TO is_subscribed;
```

Cross-References

- [01_numeric_types.md](#) — BIGINT vs UUID for primary keys
- [08_constraints.md](#) — CHECK constraints, PRIMARY KEY
- [09_sequences_identity.md](#) — BIGINT GENERATED AS IDENTITY alternative to UUID
- [../07_Indexes/](#) — partial indexes on boolean columns
- [../17_PostgreSQL_For_Backend_Engineers/](#) — UUID in APIs and distributed systems